COMPUTATIONAL BIOLOGY · MODULE 12

Sequence Alignment Methods and Applications

How two or more sequences are compared in a meaningful way — homology versus similarity, global versus local alignment, substitution matrices, the dynamic programming behind Needleman-Wunsch and Smith-Waterman, BLAST and its E-values, multiple sequence alignment, and how to read an alignment without over-trusting it.

AUTHOR	Prachya Biswas
PROGRAMME	Computer Science, BRAC University
TOPIC	Sequence Alignment Methods & Applications
SERIES	Computational Biology Meets Cells — Module 12

Contents

1. Why Alignment Matters — Homology and Similarity

2. Pairwise Alignment — Global vs Local

3. Scoring Systems — Substitution Matrices (PAM and BLOSUM)

4. Dynamic Programming — Needleman-Wunsch and Smith-Waterman

5. Heuristic Search — Understanding BLAST

6. Interpreting BLAST Output and E-values

7. Multiple Sequence Alignment Concepts

8. MSA Tools — Clustal, MUSCLE, and MAFFT

9. Alignment Visualization and Interpretation

10. Motif and Domain Identification from Alignments

11. AI-Assisted Similarity Search and Pattern Detection

12. Common Pitfalls in Alignment Analysis

13. Key Takeaways

14. What's Next

1. Why Alignment Matters — Homology and Similarity

Alignment compares DNA or protein sequences in a meaningful way. Instead of eyeballing two strings, it lines them up so that corresponding positions sit in the same column, which exposes three things:

- **Shared** regions — the same residue in both.
- **Changed** regions — a different residue at the same position.
- **Conserved** regions — stretches that stay similar across many sequences.

The slide's picture says it well: aligned sequences show clean, paired blocks, while unaligned ones are a tangle of crossing lines and question marks. Alignment replaces guesswork with a defined, repeatable comparison.

Two words that are often confused

Term	Meaning	Nature
Similarity	The observed resemblance between aligned sequences	A measurement — it comes in degrees (for example, 62% identical)
Homology	The sequences share a common evolutionary origin	A conclusion — they either do or do not; there is no "60% homologous"

Similarity is the *evidence*; homology is the *inference* drawn from it. The slide's analogy: two recipes that look alike probably came from the same original recipe, but resemblance alone does not prove it.

Two related terms, so they are not mixed up later:

- **Identity** — the residues are exactly the same.
- **Similarity (in the strict protein sense)** — the residues differ but are chemically comparable (for example, leucine and isoleucine).

> **Key idea:** alignment avoids guesswork and supports biological interpretation.

2. Pairwise Alignment — Global vs Local

Pairwise alignment compares two sequences. There are two main strategies.

	Global alignment	Local alignment
Question asked	How well do the **complete** sequences line up overall?	Which is the **best matching region**, wherever it is?
Covers	Start to end of both sequences	Only the highest-scoring stretch
Best for	Broadly similar sequences of roughly similar length	Sequences that share only a domain or motif, or differ greatly in length
Algorithm	Needleman-Wunsch	Smith-Waterman

The lesson of the slide: choose the method based on the biological question. Forcing a full-length match when only one part truly fits is a poor strategy.

A concrete contrast, computed in §4 with identical scoring:

```
Sequence 1:  CCGATTACACC
Sequence 2:    GATTACA
```

Here the second sequence is a perfect copy of a piece inside the first. A global alignment has to account for the four extra letters in sequence 1 as gaps and ends with a score of **–1**. A local alignment ignores the unmatched ends and reports the perfect 7-letter match with a score of **+7**. Same data, opposite impressions — which is why the choice matters.

3. Scoring Systems — Substitution Matrices (PAM and BLOSUM)

An alignment needs a score to decide which of many possible alignments is best. For DNA, a simple scheme is enough (match +1, mismatch –1). For proteins that is too crude, because mismatches are not equally bad:

- Replacing an amino acid with a chemically similar one may preserve function.
- Replacing it with a very different one may disrupt function.

A **substitution matrix** captures this. It gives a score to every possible pair of amino acids based on how acceptable that swap is, biologically:

- **Positive** score — the substitution is seen more often than chance would predict.
- **Negative** score — the substitution is rarer than chance.

> **Key idea:** high scores favour evolutionarily plausible matches, not just neat letter matching.

PAM vs BLOSUM

	PAM	BLOSUM
Built from	Mutations observed between **close relatives**, then **extrapolated** across evolutionary time	Substitutions observed inside **conserved blocks** of protein families
The number means	Evolutionary distance — PAM1 is one accepted point mutation per 100 residues; higher PAM means more distant	The percent identity at which sequences were clustered — BLOSUM62 uses blocks clustered at 62% identity
Higher number	More **divergent** sequences (PAM250 for distant relatives)	More **closely related** sequences (BLOSUM80 for close, BLOSUM45 for distant)

The numbers run in opposite directions, which is a common source of confusion: a high PAM number is for *distant* sequences, a high BLOSUM number is for *close* ones. **BLOSUM62** is the usual default in protein searches.

A few BLOSUM62 values to feel the logic:

Pair	Score	Why
W–W	+11	Tryptophan is rare and distinctive, so matching it is strong evidence
A–A	+4	A common residue matching is weaker evidence
I–V	+3	Both small hydrophobic residues; a conservative swap
I–L	+2	Both hydrophobic; a conservative swap
D–W	–4	Acidic versus bulky aromatic; a very different swap

Gap penalties

Scores also need a penalty for opening a gap (an insertion or deletion). Real tools normally use an **affine** gap penalty: one cost to *open* a gap and a smaller cost for each extra position it *extends*, because one long insertion is a single evolutionary event, not many.

4. Dynamic Programming — Needleman-Wunsch and Smith-Waterman

Comparing two sequences by trying every possible alignment is hopeless, because the number of alignments grows explosively. **Dynamic programming** avoids this by solving many small scoring decisions and reusing the answers.

The slide's summary: both methods fill a matrix using match, mismatch, and gap scores, then **traceback** reads the optimal alignment out of the matrix.

The recurrence

Let `F(i, j)` be the best score for aligning the first `i` letters of sequence A with the first `j` letters of sequence B. Each cell is the best of three moves:

```
F(i, j) = max(
    F(i-1, j-1) + s(a_i, b_j),    # diagonal: align a_i with b_j
    F(i-1, j)   + gap,            # up:       a_i against a gap
    F(i,   j-1) + gap             # left:     b_j against a gap
)
```

where `s` is `+match` or `–mismatch`.

Needleman-Wunsch (global)

1. Make a matrix with one extra row and column. Fill the first row and column with multiples of the gap penalty (`0, –2, –4, ...`), since aligning a prefix against nothing means all gaps.
2. Fill each remaining cell using the recurrence.
3. The **bottom-right cell** is the score of the best global alignment.
4. **Traceback** from the bottom-right to the top-left, following the move that produced each cell.

Worked example with scoring match = +1, mismatch = –1, gap = –2, aligning `GATTACA` with `GCATGCA`:

```
     -    G    C    A    T    G    C    A
 -   0   -2   -4   -6   -8  -10  -12  -14
 G  -2    1   -1   -3   -5   -7   -9  -11
 A  -4   -1    0    0   -2   -4   -6   -8
 T  -6   -3   -2   -1    1   -1   -3   -5
 T  -8   -5   -4   -3    0    0   -2   -4
 A -10   -7   -6   -3   -2   -1   -1   -1
 C -12   -9   -6   -5   -4   -3    0   -2
 A -14  -11   -8   -5   -6   -5   -2    1
```

Reading one cell: the `1` in row **G**, column **G** comes from the diagonal, `0 + 1` (a match). The `0` in row **A**, column **C** is the best of: diagonal `1 + (–1) = 0`, up `–1 – 2 = –3`, left `–1 – 2 = –3`, so `0`.

The bottom-right cell is **1**, and the traceback gives:

```
GATTACA
|  | ||
GCATGCA
```

Check by counting: 4 matches (+4) and 3 mismatches (–3) = **+1**. ✓

Smith-Waterman (local)

It differs from Needleman-Wunsch in three places:

1. No score may drop below zero: `H(i, j) = max(0, ...)`. A zero means "start a new alignment here".
2. The first row and column are all zeros.
3. The answer is the **highest cell anywhere** in the matrix, and traceback runs from there **until a zero is reached**.

Worked example: `CCGATTACACC` against `GATTACA`, same scoring:

```
     -    G    A    T    T    A    C    A
 -   0    0    0    0    0    0    0    0
 C   0    0    0    0    0    0    1    0
 C   0    0    0    0    0    0    1    0
 G   0    1    0    0    0    0    0    0
 A   0    0    2    0    0    1    0    1
 T   0    0    0    3    1    0    0    0
 T   0    0    0    1    4    2    0    0
 A   0    0    1    0    2    5    3    1
 C   0    0    0    0    0    3    6    4
 A   0    0    1    0    0    1    4    7
 C   0    0    0    0    0    0    2    5
 C   0    0    0    0    0    0    1    3
```

The highest cell is **7**, and the score climbs one step at a time along the diagonal (1, 2, 3, 4, 5, 6, 7). Traceback gives the perfect local match:

```
GATTACA
|||||||
GATTACA
```

The same pair aligned **globally** scores **–1** (the match is worth +7, the four unmatched end letters cost 4 × –2 = –8). That is the global-versus-local difference in numbers.

Cost

Both algorithms fill an `m × n` matrix, so time and memory grow as **O(m·n)**. Fine for two genes; far too slow to run against a database of billions of letters. That is the problem BLAST solves.

A runnable version

A plain-Python implementation of the recurrences above that reproduces the matrices in this section. No libraries are needed.

```
def needleman_wunsch(a, b, match=1, mismatch=-1, gap=-2):
    n, m = len(a), len(b)
    F = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        F[i][0] = i * gap
    for j in range(1, m + 1):
        F[0][j] = j * gap

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            s = match if a[i - 1] == b[j - 1] else mismatch
            F[i][j] = max(F[i - 1][j - 1] + s,   # diagonal
                          F[i - 1][j] + gap,     # up
                          F[i][j - 1] + gap)     # left

    # Traceback from the bottom-right corner
    i, j, top, bottom = n, m, "", ""
    while i > 0 or j > 0:
        s = match if i > 0 and j > 0 and a[i - 1] == b[j - 1] else mismatch
        if i > 0 and j > 0 and F[i][j] == F[i - 1][j - 1] + s:
            top, bottom, i, j = a[i - 1] + top, b[j - 1] + bottom, i - 1, j - 1
        elif i > 0 and F[i][j] == F[i - 1][j] + gap:
            top, bottom, i = a[i - 1] + top, "-" + bottom, i - 1
        else:
            top, bottom, j = "-" + top, b[j - 1] + bottom, j - 1
    return F[n][m], top, bottom


score, top, bottom = needleman_wunsch("GATTACA", "GCATGCA")
print(score)    # 1
print(top)      # GATTACA
print(bottom)   # GCATGCA
```

To turn this into Smith-Waterman, clamp every cell at zero with `max(0, ...)`, start the traceback at the highest cell, and stop when a cell reaches zero.

5. Heuristic Search — Understanding BLAST

Dynamic programming is accurate but becomes slow on very large sequence databases. **BLAST** (Basic Local Alignment Search Tool) is a *heuristic* tool built to find similar biological sequences quickly.

A heuristic trades a guarantee for speed. Smith-Waterman guarantees the optimal local alignment for the scoring scheme. BLAST does not; it gains its speed by avoiding exhaustive comparison, and can occasionally miss a weak alignment that Smith-Waterman would have found.

How BLAST works

1. **Seed.** Search for short exact (or near-exact) matches, called **words**, between the query and each database sequence.
2. **Locate.** These seeds show where a useful local alignment is likely to be.
3. **Extend.** Grow each promising seed outward into a full local alignment, stopping when the score stops being worthwhile.

The analogy: instead of comparing two books word by word, look first for a rare phrase they share, and read around only those spots.

Typical default word sizes: about 3 residues for proteins, and about 11 bases for standard nucleotide searches. A longer word is faster but less sensitive.

The BLAST family

Program	Query	Database
`blastn`	Nucleotide	Nucleotide
`blastp`	Protein	Protein
`blastx`	Nucleotide, translated in all six frames	Protein
`tblastn`	Protein	Nucleotide, translated in all six frames
`tblastx`	Nucleotide, translated	Nucleotide, translated

What it is used for

- Identifying an unknown sequence.
- Finding **homologs** of a gene or protein in other organisms.
- Inferring function from similarity.

> **Key idea:** BLAST trades exhaustive comparison for speed, and returns candidates that must still be judged.

6. Interpreting BLAST Output and E-values

A BLAST result lists matched sequences, how strong each match is, and how much of the sequence the alignment covers.

The columns that matter

Field	What it tells you
Description / accession	Which database sequence matched
Score (bits)	How good the alignment is, on a scale that does not depend on database size
E-value	How many hits this good would be expected **by chance**
Percent identity	Fraction of aligned positions that are identical
Query coverage	How much of *your* query the alignment spans
Alignment length	The length of the aligned region

The E-value

The E-value estimates how many alignments with a score at least this high you would expect to see **purely by chance** when searching a database of this size.

$E = K \cdot m \cdot n \cdot e^{(-\lambda S)}$

Here `m` is the query length, `n` is the database size, `S` is the raw score, and `K` and λ are constants of the scoring system. In terms of the bit score `S'`, this simplifies to **$E = m \cdot n \cdot 2^{(-S')}$**.

Two consequences follow straight from the formula:

1. **A higher score gives an exponentially smaller E-value.**
2. **E grows with database size.** The same alignment is less surprising in a bigger database, because there are more chances to find a lucky match.

Worked numbers, with a query of length 300 searched against a database of 10^9 letters:

Bit score	$E = 300 \times 10^9 \times 2^{(-S')}$	Reading
30	≈ 279	Hundreds of equally good hits expected by chance — meaningless
50	≈ 0.00027	Unlikely by chance — a candidate
100	$\approx 2 \times 10^{-19}$	Essentially impossible by chance

How to read it

- **Lower E-value means less likely to be random**, so more statistically significant.
- E-values near zero suggest strong hits; larger values call for more caution.
- Common working cut-offs are 10^{-5} or smaller for confident hits, but there is no universal threshold; it depends on the question.
- An E-value of exactly 1 means one equally good match is expected by chance.

Do not rely on the E-value alone

The slide is explicit: also assess alignment length and similarity. A tiny, very high-scoring stretch can have a good E-value yet cover only a sliver of the query. Percent identity can look excellent over just 20 residues. Always check **E-value, query coverage, and percent identity together**.

> **Key idea:** a meaningful BLAST hit is statistically strong and biologically relevant.

7. Multiple Sequence Alignment Concepts

Multiple sequence alignment (MSA) aligns several related sequences together, not just one query against one target. It asks how a whole group of sequences matches **position by position**: each column of the alignment is assumed to hold residues descended from one ancestral position.

What it reveals across a sequence family:

- **Substitutions** — different residues in the same column.
- **Insertions and deletions** — shown as gaps.
- **Conserved regions** — columns that stay the same across the family, which often indicate an important structural or functional role.

These patterns show both **similarity and variation** within genes or proteins. Researchers use MSA for homology, evolution (it is the usual input for building phylogenetic trees), and downstream analysis such as motif finding.

Why MSA is hard

Optimal dynamic programming on k sequences needs a k-dimensional matrix, and its cost grows exponentially with k. So practical MSA tools use heuristics. The classic one is **progressive alignment**:

1. Align every pair of sequences and measure how similar they are.
2. Build a **guide tree** from those distances.
3. Align the most similar sequences first, then add more distant ones following the tree.

A consequence: **early mistakes are never undone.** An error made when aligning the first pair stays in the final result. Modern tools add iterative refinement to reduce this.

> **Key idea:** MSA helps read biological history across many sequences at once.

8. MSA Tools — Clustal, MUSCLE, and MAFFT

Tool	Character
Clustal (ClustalW, Clustal Omega)	A classic, widely used, beginner-friendly tool
MUSCLE	Valued for accurate alignments produced efficiently
MAFFT	Especially useful for larger sequence datasets

All three perform multiple sequence alignment in practice. **Tool choice depends on the task, dataset size, and processing efficiency needs.** There is no single best tool, and when a result matters it is reasonable to compare the output of more than one.

> **Key idea:** these tools turn sequence comparison into usable alignments.

9. Alignment Visualization and Interpretation

An alignment is read **column by column**. Three kinds of column carry meaning:

Column type	What it often means
Conserved (same residue throughout)	Functionally or structurally important; change was not tolerated
Variable (many different residues)	Flexibility, adaptation, or a region under little constraint
Gap	An insertion or deletion, or an evolutionary event

Colour coding (conserved, variable, and gap in the slide's legend) makes conservation and similar residues easy to spot at a glance. Viewers such as Jalview colour residues by their chemical class, so a column of different-but-similar residues still looks coherent.

A summary of one column is often written as a **consensus** sequence, taking the most common residue at each position.

> **Key idea:** interpret alignments as biological evidence, not just neat software output.

10. Motif and Domain Identification from Alignments

Alignments can reveal biologically meaningful patterns:

Term	Meaning	Example of what it does
Motif	A short conserved pattern	Binding a metal ion, binding DNA, catalysis
Domain	A larger structural or functional unit within a protein	A kinase domain, an SH2 domain

A repeated, conserved pattern across an alignment is a clue worth investigating. **Highly conserved residues often mark regions important for structure or function**, because evolution has repeatedly rejected changes there.

The reverse is just as useful. **Variable regions may support functional differences** between related proteins, such as one family member recognising a different partner.

The question the lecture frames is: *which parts actually do the biological heavy lifting?* Conservation is the first, strong hint; experiments and databases (for example, Pfam and PROSITE record known domains and motifs) provide the confirmation.

11. AI-Assisted Similarity Search and Pattern Detection

AI extends motif and domain analysis into similarity search and pattern detection:

- It scans very large datasets and finds recurring or **subtle similarities missed manually**.
- It is especially useful for **noisy, numerous, or only partially similar** sequences, where simple alignment struggles.
- Models learn known motif, domain, and functional features and **flag new matches**.

The limit is stated just as clearly: detected matches may suggest shared function or evolution, but they are **only clues**. AI improves speed and scale; **scientists still make the final interpretation**. In the slide's picture, the AI flags a pattern and a person writes the interpretation.

> **Key idea:** a model can say "this looks like a known pattern"; it cannot establish that the pattern means the same thing in a new context.

12. Common Pitfalls in Alignment Analysis

Question alignments critically, **even after AI-assisted search**.

Pitfall	Why it misleads	What to do
A high score is read as shared function	Similar sequence does not guarantee identical function; paralogs often diverge	Check known biology and the annotation evidence (see Module 11, UniProt)
Short similar regions	A short match can occur by chance	Check E-value *and* coverage; look at the length

Pitfall	Why it misleads	What to do
Many gaps	The aligner may be forcing a weak link between sequences that are not really related	Inspect the gap pattern; question the homology claim
Repetitive or low-complexity regions	Stretches like QQQQQQ or AAAAAA match each other and inflate the score	Use low-complexity filtering (SEG for protein, DUST for nucleotide)
Wrong scoring choice	A matrix tuned for the wrong evolutionary distance gives poor alignments	Match the matrix to how related the sequences are
Ignoring context	An alignment tells you nothing about which organism or family it belongs in	Interpret results using species, gene family, and known biology

Key idea: an alignment is evidence. Whether it supports a conclusion depends on the biology around it.

13. Key Takeaways

- Similarity is measured; homology is inferred. A sequence is not "60% homologous".
- Global alignment (Needleman-Wunsch) fits whole, similar-length sequences; local alignment (Smith-Waterman) finds the best shared region. Choose by the biological question.
- Substitution matrices score amino acid pairs by biological plausibility. A high PAM number is for distant sequences; a high BLOSUM number is for close ones. BLOSUM62 is the common default.
- Dynamic programming fills a matrix from match, mismatch, and gap scores, and traceback reads out the optimal alignment. It costs O(m·n), which is why it is too slow for whole databases.
- BLAST is a heuristic: seed, then extend. It is fast but not guaranteed optimal.
- E-value is the number of equally good hits expected by chance. Lower is better, it grows with database size, and it must be read together with coverage and identity.
- MSA aligns many sequences column by column; progressive methods build on a guide tree, and early errors persist.
- Conserved columns point to important residues; variable columns point to flexibility.
- AI scales the search, but its matches are clues that a scientist has to interpret.

14. What's Next

With the ability to compare sequences, the next step is to use alignments to ask evolutionary questions: how related are these sequences, and in what order did they diverge? That leads to phylogenetic analysis, which builds trees from the multiple sequence alignments produced here.